Words Are Not Motions: Steering Vision-Language-Action Models with Jacobian Pullbacks

Qinhe Peng
University of Pennsylvania
pengqh20@engineering.upenn.edu

Dinesh Jayaraman
University of Pennsylvania
dineshj@engineering.upenn.edu

Subin Kim
University of Pennsylvania
kim34@engineering.upenn.edu

Abstract: Just as humans form an intention before committing muscles to movement, vision-language-action (VLA) models must bridge high-level semantic reasoning with low-level physical control, yet this translation remains opaque. Token-based steering methods, carried over from language-model interpretability, presume that semantic representations naming motion concepts execute them. Instead, we introduce Jacobian Kinematic and Semantic Steering (JacKSS), a training-free framework that pulls downstream target policy outputs back into internal hidden representations via gradient-based Jacobian metrics. By formulating objectives over both vocabulary logits and 3D end-effector Cartesian positions, JacKSS identifies both semantic and kinematic subspaces, showing that co-activating overlapping components enhances behavioral control. Across discrete autoregressive and continuous flow-matching policies in both simulation and real-world trials, JacKSS achieves zero-shot control for speed, direction, smoothness and semantic object choices. Project website: jacobianvla.github.io

Keywords: Vision-Language-Action Models, Mechanistic Interpretability, Activation Steering

1 Introduction

Vision-Language-Action (VLA) models offer a compelling paradigm for generalist robot manipulation, yet how high-level semantic instructions translate into low-level kinematic commands remains opaque. Existing interpretability and steering methods rely on external proxies: token-based approaches steer representations via vocabulary logits [1], implicitly assuming that the features naming a motion concept such as "up" or "slow" are the ones executing it. Using linear probes trained on labeled robot states [2, 3] and dictionary-learning methods through sparse autoencoders (SAE) [4, 5, 6] require costly auxiliary training without explaining downstream motor execution. Because robots are embodied agents whose actions unfold in physical space, reliably steering robot behavior requires representations tied to physical control, unlike LLMs which only require linguistic proxies.

To bridge this gap, we introduce Jacobian Kinematic and Semantic Steering (JacKSS), a training-free framework that maps target objectives directly back into internal hidden representations via gradient-based Jacobian metrics. In a single backward pass through the policy's native downstream dynamics, JacKSS extracts layer-wise steering directions for both high-level semantic concepts and physical kinematic behaviors. At inference time, JacKSS intervenes on hidden states through a contrastive residual swap that selectively replaces unwanted source behaviors with intended target directions without altering model weights. Evaluated across discrete autoregressive policies and continuous flow-matching experts, JacKSS achieves zero-shot steering over kinematic concepts and

10th Conference on Robot Learning (CoRL 2026), Austin TX, USA.

semantic object selections in both simulation and real-world trials while unveiling the geometric disentanglement between semantic and motor subspaces. Our contribution is as follows:

- We Derive causally grounded steering vectors via native pullbacks with jacobian metrics without auxiliary training such as SAEs, linear probes, or manual neuron curation.
- We Demonstrate zero-shot behavioral steering across discrete and continuous VLA architectures in both simulated rollouts and real-world tabletop manipulation.
- We Characterize the geometric disentanglement between semantic and motor subspaces, showing that activating overlapping subspaces outperforms isolated semantic or kinematic steering.

2 Jacobian Pullback: Steering Semantic and Kinematic Subspaces

JacKSS controls policy behavior in two stages: first, it constructs a Jacobian metric via gradient pullbacks to map target output objectives into hidden-state directions across layers Second, it steers inference at run time by applying a training-free residual swap between contrastive directions to achieve the target behavior without altering model weights.

Constructing Jacobian Pullback. Let $h^{(\ell)} \in \mathbb{R}^{P\times d}$ stack the residuals $h_p^{(\ell)}$ at the P prefix positions immediately *after* transformer block $\ell \in \{0,\dots,L-1\}$. For a given single observation per timestep, let F be a scalar output from a scoring function for the target concept, and $\mathcal{S} \subseteq \{1,\dots,P\}$ a set of source index. Introduce one perturbation $\delta \in \mathbb{R}^d$ shared across those positions after block ℓ. With all edits initialized to zero, the layer-specific gradient is

$$g_{\mathcal{S}}^{(\ell)} = \nabla_\delta F\big(h^{(\ell)} + \mathbf{1}_{\mathcal{S}}\delta^\top\big)\Big|_{\delta=0} = \sum_{p\in\mathcal{S}} \Big(\frac{\partial z}{\partial h_p^{(\ell)}}\Big)^\top \nabla_z F, \qquad \hat{g}^{(\ell)} = \frac{\bar{g}^{(\ell)}}{\|\bar{g}^{(\ell)}\|_2}, \tag{1}$$

where $\bar{g}^{(\ell)}$ averages the raw gradient over N observation frames, and z denotes the downstream model outputs evaluated by the scoring function F below. Through this pullback formulation, the transpose Jacobian $(\partial z/\partial h_p^{(\ell)})^\top$ acts as a bridge translating desired directional variations defined directly in the target output space noted below into corresponding alignment directions in the internal hidden representation space at every layer (App. B.2).

Formulating Subspace-Specific Target. For *Semantic Target,* we set a concept c such as direction or speed, and use a single-token vocabulary set $\mathcal{T}$ consisting of specific target vocabularies— such as "fast" for speed— and a background vocabulary set $\mathcal{C}$ consisting of vocabularies for covering a broader range of semantic concepts. We define

$$r_c = \frac{1}{|\mathcal{T}|}\sum_{i\in\mathcal{T}} E_i - \frac{1}{|\mathcal{C}|}\sum_{j\in\mathcal{C}} E_j, \qquad F^{\text{sem}} = \frac{1}{|\mathcal{V}|}\sum_{p\in\mathcal{V}} r_c^\top \,\text{RMSNorm}_{\text{final}}(h_p^{(L-1)}), \tag{2}$$

where E_i is the output embedding of token i and $\mathcal{V}$ the valid prefix positions. Subtracting the mean embedding of $\mathcal{C}$ centers the representation, isolating preference for the target concept over generic semantic background rather than raw activation magnitude (App. B.1). For *Kinematic Target,* we explicitly define 3D end-effector Cartesian positions as the target output defined as below:

$$F^{\text{up}} = \frac{1}{H}\sum_{t=1}^{H}[D_t]_3, F^{\text{fast}} = \frac{1}{H\Delta t}\sum_{t=1}^{H}\|D_t\|_2, F^{\text{jerky}} = \frac{1}{(H-2)\Delta t^3}\sum_{t=2}^{H-1}\|D_{t+1}-2D_t+D_{t-1}\|_2. \tag{3}$$

Where $D_t \in \mathbb{R}^3$ denote the command-derived EEF displacement for horizon index t of an H-step chunk (App. B.2). For *Shared Target of Semantic and Kinematic concepts,* we combine the two unit directions, $\hat{g}_{\text{shared}}^{(\ell)} = \frac{\hat{g}_{\text{sem}}^{(\ell)}+\hat{g}_{\text{kin}}^{(\ell)}}{\|\hat{g}_{\text{sem}}^{(\ell)}+\hat{g}_{\text{kin}}^{(\ell)}\|_2}$, which co-activates the concept and the command (App. B.2).

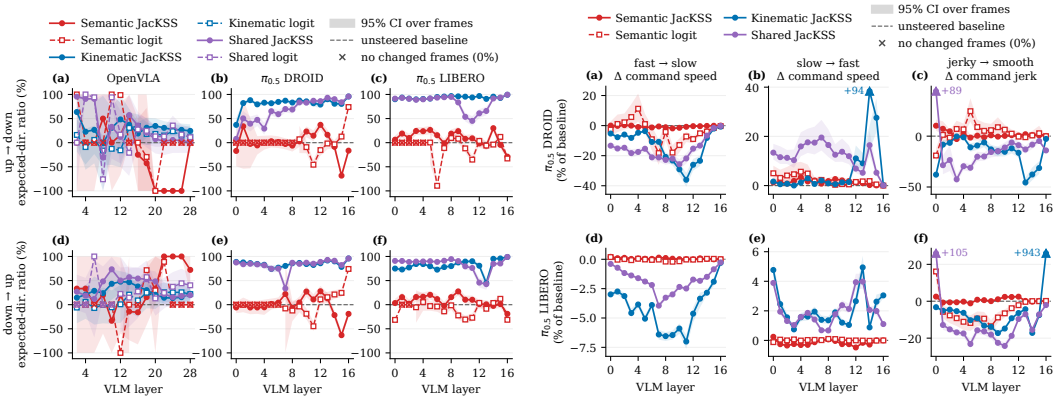


Figure 1: **Open-loop steering. Left:** expected-direction ratio of the steered-minus-baseline velocity command for up→down (a–c) and down→up (d–f) swaps in OpenVLA LIBERO (a, d), $\pi_{0.5}$ DROID (b, e), and $\pi_{0.5}$ LIBERO (c, f). **Right:** percent change of mean commanded speed for fast→slow (a, d) and slow→fast (b, e) and of within-chunk jerk for jerky→smooth (c, f) in $\pi_{0.5}$ DROID (a–c) and LIBERO (d–f). Each point is one single-layer swap at strength 1 against the paired unedited prediction (dashed); bands are 95% bootstrap intervals over frames (App. D.1).

Residual Swap Steering. Given unit target concept direction $\hat{g}_t^{(\ell)}$ and contrastive concept direction $\hat{g}_s^{(\ell)}$ at layer ℓ, we edit every position $p \in \mathcal{S}$ by

$$h'_p = h_p + \left[\langle h_p, \hat{g}_s^{(\ell)} \rangle\right]_+ (\hat{g}_t^{(\ell)} - \hat{g}_s^{(\ell)}) \quad . \tag{4}$$

Intuitively, this edit replaces any positive component of the source direction $\hat{g}_s^{(\ell)}$ in h_p with the target direction $\hat{g}_t^{(\ell)}$, acting as an exact no-op when the source concept is absent.

3 Experiments

We evaluate JacKSS on $\pi_{0.5}$ (LIBERO and DROID) and OpenVLA (LIBERO) using single-layer residual swaps following equation 4, comparing kinematic, semantic, and shared pullback directions clarified in equation 2 and equation 3 against the static semantic logit baseline and prompt adverbs (App. C.1, D).

3.1 Kinematic-Space Steering

Vertical Direction. We evaluate single-layer up↔down swaps on held-out frames under matched noise (Fig. 1, left). In $\pi_{0.5}$ LIBERO, kinematic pullbacks achieve a 93.8% expected-direction ratio and shared directions reach 83.7%, whereas semantic JacKSS (10.1%) and semantic logits (-9.2%) largely fail to steer motion, a trend mirrored across embodiments in DROID. In OpenVLA, while semantic JacKSS can steer effectively at specific depths, its performance is strongly layer-dependent and asymmetric between polarities. Across both architectures, physically grounded pullbacks provide consistent, polarity-robust directional control where purely linguistic features remain brittle.

Speed and Smoothness. In open loop (Fig. 1, right), kinematic and shared directions produce substantial speed and jerk shifts across layers, while semantic methods induce less change. In closed-loop LIBERO episodes (Tab. 1), kinematic fast→slow reduces measured speed by 5.2% while preserving task success rate above the Base $\pi_{0.5}$ success rate of 92%, and shared jerky→smooth reduces jerk by 9.8%, whereas prompt and semantic conditions remain indistinguishable from baseline. On real DROID hardware (Fig. 2, left), shared steering accelerates mean EEF speed by 38% and dampens roughness by up to 27%, whereas adverb prompts fail to produce intended speedups. We hypothesize that while text prompts and static embeddings sometimes get ignored by the policy's low-level kinematic execution, shared pullbacks succeed because they simultaneously co-activate semantic intent with downstream velocity commands.

Method	Slowing (fast→slow)		Speeding up (slow→fast)		Smoothing (jerky→smooth)	
	Succ. (%)	Δspeed (%) [95% CI]	Succ. (%)	Δspeed (%) [95% CI]	Succ. (%)	Δjerk (%) [95% CI]
Base $\pi_{0.5}$	92	—	92	—	92	—
Prompt	94	$+1.2$ $[-1.3, +3.7]$	91	-0.2 $[-2.7, +2.2]$	95	$+0.7$ $[-2.2, +3.5]$
Semantic logit	91	-1.4 $[-3.3, +0.5]$	92	-0.7 $[-2.6, +1.2]$	93	-1.7 $[-5.1, +1.9]$
Semantic JacKSS	96	$+0.7$ $[-1.0, +2.7]$	95	$+0.3$ $[-1.4, +2.1]$	92	$+4.5$ $[-0.3, +9.8]$
Kinematic JacKSS	95	$\mathbf{-5.2}$ $[-7.4, -3.0]$	92	$+1.0$ $[-1.7, +3.6]$	93	-4.7 $[-8.3, -0.5]$
Shared JacKSS	94	-1.9 $[-4.7, +0.6]$	91	$\mathbf{+2.9}$ $[-0.4, +5.9]$	91	$\mathbf{-9.8}$ $[-13.7, -4.9]$

Table 1: **Closed-loop steering.** We steer $\pi_{0.5}$ LIBERO-10 for speed and smoothness. Changes are mean paired percentages of measured EEF speed or jerk relative to the matched baseline, with 95% bootstrap intervals over trial pairs; the prompt appends the corresponding adverb. Bold marks the best point estimate per goal (App. D.2).

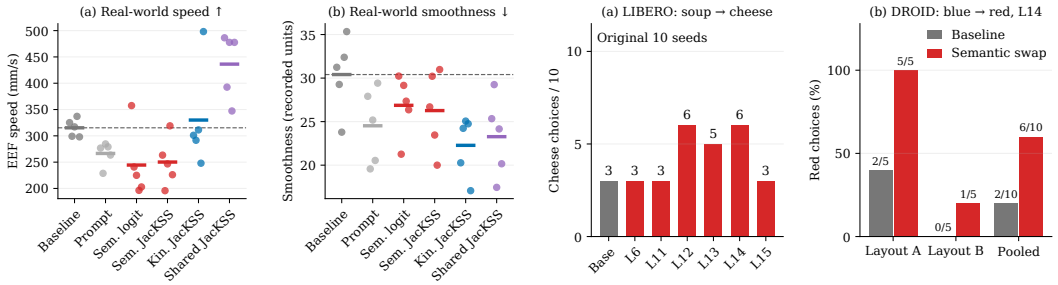


Figure 2: **Left: Real-world DROID motion steering.** ($n = 5$ per condition, layer 10) (a) EEF speed in mm/s and (b) the recorded smoothness metric; dots are trials, marks are means, dashed lines the baseline mean. **Right: Object choice under a fixed instruction.** (a) cheese choices out of ten matched seeds for single-layer soup→cheese swaps in the LIBERO-10 task 1 and task 2 (Tab. 2, App. D.3). (b) red-cube choices on the real robot for a blue→red semantic JacKSS swap at layer 14, five trials per layout.

3.2 Semantic-Space Steering

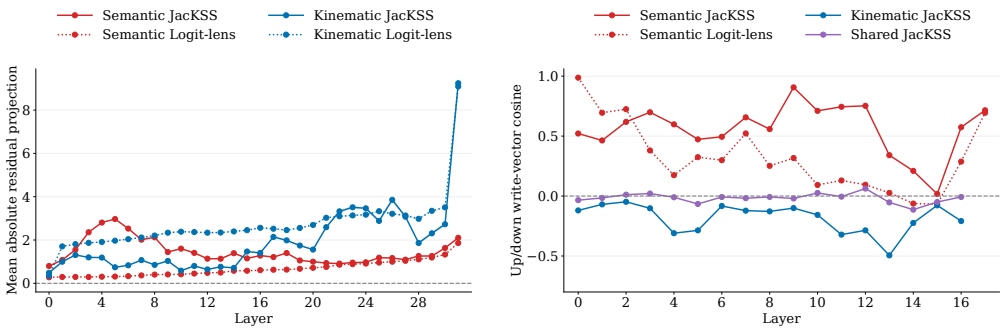
Because categorical decisions lack a differentiable kinematic formulation, we steer discrete object and color choices purely through semantic-space pullbacks under fixed instructions. In LIBERO-10 task (App. D.3), a soup→cheese swap at layers 12 or 14 doubles cheese selections from 3/10 to 6/10 while maintaining 100% task success (Fig. 2, right (a)). Similarly, on a real robot executing "pick the cube into the bowl", a blue→red swap at layer 14 increases red cube selections from 2/10 to 6/10 across mirrored layouts (Fig. 2, right (b)). This demonstrates a clear functional division of labor in the representation space: while kinematic pullbacks govern *how* the arm executes motion, the semantic subspace uniquely controls *what* target the robot interacts with.

3.3 Mechanistic Disentanglement of Semantic and Kinematic Subspaces

Examining the unedited representations across depth, geometry, and time reveals the structural basis for this functional division (Fig. 3): **Layer Depth.** OpenVLA residual projections show kinematic directions emerging sharply in the final layers near the action decoder while semantic projections remain flat, reflecting the progressive transformation of abstract goals into actionable motor commands. **Feature Geometry.** In $\pi_{0.5}$, top-neuron write vectors for "up" and "down" are positively aligned for semantic selections ($+0.5$) but have negative correlation for kinematic selections (-0.19). Because language models represent antonyms as conceptually related neighbors whereas motor systems must represent them as opposing kinematic actions, steering with word-aligned directions may produce weak or inverted kinematic responses. **Temporal Dynamics.** Along executed demonstrations, kinematic and shared neurons tightly phase-lock to instantaneous kinematic lifts and descents, while semantic neurons remain broadly active to maintain invariant task context throughout the rollout (Figs. 4, 5).

4 Conclusion

We introduced JacKSS, a training-free framework that extracts causally grounded steering vectors by pulling differentiable action objectives back through a VLA model's native downstream computation. Across simulated benchmarks and physical hardware, motor-grounded directions reliably



(a) Residual projections onto up directions (OpenVLA, LIBERO).

(b) Up/down write-vector cosine per layer ($\pi_{0.5}$ LIBERO).

Figure 3: **Analysis of Semantic and Kinematic Subspaces.** (a) Mean absolute projection of unedited residuals (100 observations, bands ± 1.96 SE): kinematic JacKSS directions are strongly expressed only near the output. (b) Cosine between the mean write vectors of the top-ten up and top-ten down neurons: semantic selections are aligned, physical selections are opposed, shared selections near orthogonal (App. E).

control kinematic direction, speed, and smoothness, while word-aligned directions uniquely steer categorical object choice. Our mechanistic findings support this functional division of labor, in which embodied robot control requires representations grounded in physical action dynamics alongside linguistic proxies.

References

[1] B. Häon, K. Stocking, I. Chuang, and C. Tomlin. Mechanistic interpretability for steering vision-language-action models. *arXiv preprint arXiv:2509.00328*, 2025. URL https://arxiv.org/abs/2509.00328.

[2] H. Lu, H. Li, P. S. Shahani, S. Herbers, and M. Scheutz. Probing a vision-language-action model for symbolic states and integration into a cognitive architecture. *arXiv preprint arXiv:2502.04558*, 2025. URL https://arxiv.org/abs/2502.04558.

[3] M. Molinari, L. Nevali, S. Navani, and O. G. Younis. Emergent world representations in OpenVLA. *arXiv preprint arXiv:2509.24559*, 2025. URL https://arxiv.org/abs/2509.24559.

[4] M. A. Khan, N. Boškov, F. M. Anwar, and M. A. Khan. Controlling vision–language–action policies through sparse latent directions. In *Mechanistic Interpretability Workshop at NeurIPS*, 2025. URL https://openreview.net/forum?id=wtf3ww1EOL.

[5] A. Swann, L. McGranahan, H. Buurmeijer, M. Kennedy III, and M. Schwager. Sparse Autoencoders Reveal Interpretable and Steerable Features in VLA Models. *arXiv preprint arXiv:2603.19183*, 2026. URL https://arxiv.org/abs/2603.19183.

[6] H. Buurmeijer, C. Amo Alonso, A. Swann, and M. Pavone. Observing and Controlling Features in Vision-Language-Action Models. *arXiv preprint arXiv:2603.05487*, 2026. URL https://arxiv.org/abs/2603.05487.

[7] H. J. Jeong, G. Swamy, and A. Bajcsy. Learning What to Say to Your VLA: Mostly Harmless Vision Language Action Model Steering. *arXiv preprint arXiv:2606.12299*, 2026. URL https://arxiv.org/abs/2606.12299.

[8] T. D. Trinh, N. Akhtar, and B. Azam. Altered Thoughts, Altered Actions: Probing Chain-of-Thought Vulnerabilities in VLA Robotic Manipulation. *arXiv preprint arXiv:2603.12717*, 2026. URL https://arxiv.org/abs/2603.12717.

[9] W. Gurnee, N. Sofroniew, A. Pearce, M. Piotrowski, I. Kauvar, R. Chen, A. Soligo, P. Bogdan, E. Ong, R. Wang, B. Thompson, D. Abrahams, S. Kantamneni, E. Ameisen, J. Batson, and J. Lindsey. Verbalizable representations form a global workspace in language models. *Transformer Circuits Thread*, 2026. URL https://transformer-circuits.pub/2026/workspace/index.html.

A Related Work

Recent work explores interpreting and steering vision-language-action (VLA) models through their internal representations across three key paradigms: Methods that *probe internal representations* deploy linear and nonlinear readouts to recover symbolic object and action states [2] or track latent state transitions [3]. However, these observational approaches do not provide causal intervention mechanisms for direct behavioral control. Another line of work operates through *feature dictionaries and activation interventions*, where sparse autoencoders (SAEs) isolate reusable motion primitives and contrastive latent directions for policy control [4, 5], Häon et al. [1] modulate feed-forward value vectors aligned with target concepts, and Buurmeijer et al. [6] train auxiliary linear observers to derive minimum-norm activation edits. Nevertheless, these strategies require costly SAE training, manual neuron curation, or auxiliary supervised readouts, while leaving downstream computation to action outputs largely unexploited. Other approaches steer frozen policies via external language feedback [7] or disrupt execution by perturbing entity references in chain-of-thought traces [8]. While effective for semantic guidance, they offer limited control over low-level motor primitives. Unlike prior techniques, JacKSS leverages the model's native downstream computation via vector-Jacobian pullbacks through both backbone layers and continuous action experts. This enables zero-training, causally grounded steering of disentangled semantic and motor subspaces without auxiliary decoders, SAEs, or manual feature selection.

B Implementation and protocol provenance

Vocabulary and calibration versions. The pipeline configures a unified 143-token center for new $\pi_{0.5}$ LIBERO/DROID semantic fits and a 384-token center for new OpenVLA fits. Target IDs and center hashes are explicit in the configuration.

Data construction and separation. The release's LIBERO calibration selects 40 fitting observations per task from a 1900-frame pool and preserves deterministic frame identities. Original and additional evaluation batches each contain 100 observations from different demonstrations, disjoint from fitting demonstrations. DROID fitting uses four observations from each of 100 episodes; its 100 evaluation frames are distinct frames from these same episodes.

Two closed-loop randomization protocols. Adverb rollouts use matched initial states and noise seeds derived from a hash of the trial seed and replanning index. Wildcard object trials use seeded native resets and native policy RNG splitting. Both predict ten commands and execute five before replanning, with ten settling steps and a 520-step limit.

B.1 S2/S3

S2: restricted source positions. For $\pi_{0.5}$, S2 applies the shared perturbation only at the final valid prefix position. For OpenVLA, S2 includes the final prompt position and the causally available preceding action positions. Its semantic objective is differentiated at each of seven greedy action-prediction steps and averaged over $1 + \cdots + 7 = 28$ source–target pairs; the preceding discrete action tokens are held fixed during differentiation. At inference, S2 edits the final prompt residual and each new cached decode position.

S3: all valid prefix positions. For $\pi_{0.5}$, S3 perturbs every valid image/text prefix position and excludes padding. Semantic output readouts average over valid positions. Changing source scope therefore changes the fitted sensitivity. The current release uses OpenVLA S2 and $\pi_{0.5}$ S3. Fit and steering scopes always match. The action expert receives no direct residual edit in these prefix experiments.

OpenVLA semantic fitting computes the centered semantic logit readout at each of seven prediction positions, and accumulates derivatives over the available S2 source positions. The 28-pair normalization is applied before frame averaging. Its physical readout is the centered mean logit over the relevant vertical action-bin set at certain concept-related coordinate index, for example for "up" and "down" it should be coordinate index 2, with positive/negative bin groups specified by the tokenizer

configuration. Each polarity is centered against the reference vocabulary; it is not the difference between the positive and negative bin means. The derivative averages the three causal source positions available at the dz step. In particular, the exact-negative physical construction used for $\pi_{0.5}$ should not be assumed for these separately centered OpenVLA objectives.

B.2 Physical objectives, DROID command geometry, and the shared-neuron rule

For $\pi_{0.5}$ we fix the sampling noise, run the ten Euler steps of the flow expert on the perturbed prefix, unnormalize the action chunk, and convert it to end-effector displacements: Cartesian commands in LIBERO and forward kinematics of joint commands in DROID. Let $D_t \in \mathbb{R}^3$ be the command-derived EEF displacement at horizon index t of an H-step chunk. where each prediction is referenced independently to the observed q rather than integrated. The three motor objectives differentiated in Section 2 are

B.3 Fitting and deployment procedure

For each checkpoint and source scope, we fix the calibration observations, token targets, centering vocabulary and initial action-sampling noise. We introduce independent zero-valued residual edit variables at each layer, evaluate the selected scalar objective, and differentiate it with respect to these variables. Gradients are accumulated across observations before per-layer normalization. Semantic and physical directions are normalized independently before constructing their shared direction.

For each evaluated condition, source and target vectors must have the same checkpoint, layer, residual width and source-scope convention.

C Methodological Details and Mathematical Formulations

C.1 From the logit lens to the Jacobian lens

The Jacobian lens characterizes an intermediate representation through its average first-order influence on downstream states [9]. Let $h_{\ell,p}(x)$ be the residual at layer ℓ and source position p in context x, and let $h_{\text{final},q}(x)$ be the final residual at a causally accessible position $q \geq p$. The original formulation defines

$$J_\ell = \mathbb{E}_{x,p,q\geq p}\left[\frac{\partial h_{\text{final},q}(x)}{\partial h_{\ell,p}(x)}\right] \in \mathbb{R}^{d\times d}. \tag{5}$$

This matrix summarizes how perturbations at layer ℓ propagate to the output representation across contexts and positions. Using the vocabulary matrix E defined above, the corresponding lens readout is

$$\text{JLens}_\ell(h) = \text{softmax}(E\ \text{Norm}_{\text{final}}(J_\ell h))\,. \tag{6}$$

The map J_ℓ accounts for transformations between layers before the model's output normalization and vocabulary projection are applied. Setting $J_\ell = I$ recovers a direct logit-lens readout. At the linear projection level, the direction associated with vocabulary token v is the transpose of row v of EJ_ℓ:

$$j_{\ell,v} = J_\ell^\top E_v, \qquad j_{\ell,v}^\top h = E_v^\top J_\ell h, \tag{7}$$

where E_v is written as a column vector. These linear scores precede the normalization and softmax in Eq. equation 6.

We adapt this principle to robot observations and scalar semantic or action objectives. Instead of materializing a full Jacobian matrix, we compute vector–Jacobian products for the chosen objectives. Our pullbacks include downstream nonlinearities and output normalization in the differentiation graph, producing a direction for each layer and concept (Section 2). They estimate local sensitivity; a finite intervention need not induce the behavior suggested by its linguistic label.

D Evaluation protocols and supplementary results

D.1 Open-loop aggregation and uncertainty

An open-loop sample is a fixed observation evaluated by both the unedited and intervened policy, with matched initial sampling noise. Let $D_{n,t}^{(m)}$ denote the command-derived EEF displacement at

Condition	n	Success	Cheese	Soup	Other/none
Baseline (matched ten seeds)	10	10	3	7	0
Original / S2_layer11	10	10	3	7	0
Original / S2_layer15	10	10	3	7	0
Original / S2_layer6	10	9	3	7	0
Original / S3_layer11	10	10	3	7	0
Original / S3_layer12	10	10	6	4	0
Original / S3_layer13	10	9	5	4	1
Original / S3_layer14	10	10	6	4	0
Original / S3_layer15	10	10	3	7	0
Original / S3_layer2	10	10	5	4	1
Original / S3_layer4	10	9	5	4	1
Original / S3_layer6	10	10	3	7	0

Table 2: Single-layer soup-to-cheese simulations on the original ten matched seeds. Choice is the lifted-object label, replaced by the basket occupant on success; success uses the task predicate.

horizon index t, for observation n and condition m, and let $m = 0$ denote baseline. The horizon is one for OpenVLA, ten for $\pi_{0.5}$ LIBERO, and fifteen for $\pi_{0.5}$ DROID. The displacement conversion follows Appendix B.2. In particular, the DROID conversion evaluates each predicted joint command relative to the observed joint state; it is not an integrated physical rollout.

D.2 Closed-loop LIBERO motion evaluation

The primary closed-loop table evaluates ten tasks with ten matched trials per task and condition. Residual swaps use S3 at layer 10, strength one, and all valid prefix positions. Prompt controls append the corresponding adverb without a residual intervention.

D.3 Complete single-layer object-choice results

The simulation task uses the LIVING_ROOM_SCENE2 for Task 0 and Task 1 in LIBERO-10. While the original prompt for Task 0 is "Put both the alphabet soup and the tomato sauce in the basket." and the original prompt for Task 1 is "Put both the cream cheese box and the butter in the basket.", we use prompt "Put * in the basket." for both Tasks for fair comparison. Success accepts a designated object in the basket. The recorded choice is the lifted-object label, replaced by the basket occupant when the task succeeds. Choice counts and success counts are consequently distinct: an unsuccessful trial can still have an identified object choice.

Table 2 includes only the original ten matched seeds, single-layer interventions, and the single soup→cheese swap. The baseline is restricted to those same ten seeds. S2 and S3 are listed separately, and no multi-layer, dual-swap, other-target, or additional-seed conditions enter this analysis. Directions come from the noun fit. The unedited policy picks cheese in 3/10 trials; swaps at layer 12 or 14 raise this to 6/10 with 10/10 success, while layers 6, 11, and 15 leave it at 3/10, so the effect depends on the layer. The tested layers share seeds and are exploratory comparisons; the favorable layer-12 and layer-14 results are not independent replications or evaluations on an additional held-out layer-selection set.

For the real-world task, the instruction is "pick the cube into the bowl." The source workbook records object colors for two layouts under baseline and a layer-14 blue→red semantic JacKSS swap. Table 3 preserves those sequences. Trial numbers express recording order within each layout; no matched-noise or trial-pairing assumption is made. The increase in red choices is descriptive and does not provide a separate placement-success measure.

D.4 Real-world motion measurements

Table 4 reports every value in the fast and smooth tabs of the experiment workbook. The five-trial means are computed independently for each condition. Relative effects compare each condition mean with its corresponding baseline mean. Shared steering raises mean EEF speed from 315 to 436 mm/s (+38%) and kinematic to 330 mm/s (+5%), while the "fast" prompt and both semantic

Layout	Condition	Recorded choices	Red choices
Red–bowl–blue	Baseline	R B B R B	2/5
Red–bowl–blue	Semantic JacKSS	R R R R R	5/5
Blue–bowl–red	Baseline	B B B B B	0/5
Blue–bowl–red	Semantic JacKSS	R B B B B	1/5
Both layouts	Baseline	—	2/10
Both layouts	Semantic JacKSS	—	6/10

Table 3: All recorded real-world object choices. R and B denote red and blue. Each layout contributes five observations per condition; steering swaps blue toward red at layer 14.

Experiment	Condition	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	Mean	Δ (%)
Fast	Baseline	325.12	299.18	317.04	337.01	298.02	315.274	+0.0
Fast	Prompt	276.76	228.72	284.61	279.04	263.47	266.520	-15.5
Fast	Sem. logit	357.47	240.75	224.94	196.36	202.84	244.472	-22.5
Fast	Sem. JacKSS	263.32	195.76	246.64	319.02	226.09	250.166	-20.7
Fast	Phys. JacKSS	301.06	291.58	311.27	247.97	498.15	330.006	+4.7
Fast	Shared JacKSS	486.34	392.58	477.68	347.26	477.65	436.302	+38.4
Smooth	Baseline	31.24	29.27	23.79	32.39	35.34	30.406	+0.0
Smooth	Prompt	27.907	19.57	25.18	20.54	29.42	24.523	-19.3
Smooth	Sem. logit	30.23	21.26	29.16	27.35	26.36	26.872	-11.6
Smooth	Sem. JacKSS	26.69	30.21	23.46	20	30.98	26.268	-13.6
Smooth	Phys. JacKSS	20.27	24.23	25.06	24.75	17.07	22.276	-26.7
Smooth	Shared JacKSS	25.34	29.25	17.45	24.16	20.16	23.272	-23.5

Table 4: All recorded real-world motion measurements ($n = 5$ per condition). Fast values are EEF speed in mm/s; smooth values retain the workbook's unspecified metric units. Means use the original precision; Δ is the percentage change of the condition mean from its experiment's baseline mean. Column numbers identify workbook order and do not assert paired trials. Residual conditions use layer 10.

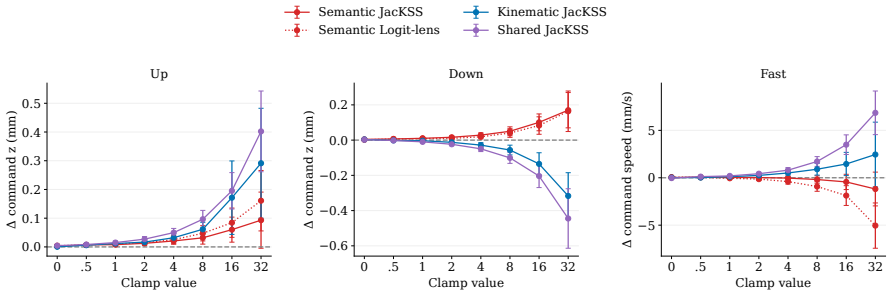


Figure 4: **Single-neuron clamp sweeps at layer 6 (S3) in $\pi_{0.5}$ LIBERO.** One selected neuron per condition is overwritten with the clamp value at every valid prefix token; the y-axis is the paired change in the predicted command on 75 held-out demonstrations (mm for up and down, mm/s for fast), with approximate 95% intervals. At clamp 32 the shared neuron gives the largest intended change for every concept, physical follows, and semantic-only neurons are weak or wrong-signed for down and fast. Clamp 0 silences the neuron and is not the unedited baseline (dashed).

conditions are slower than baseline; the recorded smoothness metric drops by 27% for kinematic and 24% for shared steering against 19% for the prompt.

E Supplementary neuron analysis and interpretation

E.1 Single-neuron clamp experiment

Causality. Clamping one selected layer-6 neuron (Fig. 4) gives, at clamp 32, the largest intended command change for the shared selection on every concept ($+0.40 \pm 0.14$ mm up, -0.45 ± 0.17 mm down, $+6.9 \pm 2.3$ mm/s fast), with kinematic next and semantic-only weaker or wrong-signed for down and fast. The ordering matches the residual-swap results at the level of single units. Together, the four analyses support complementary semantic and kinematic subspaces as the basis for steering, without establishing a complete causal circuit from language to motion.

The layer-6 S3 experiment in Fig. 4 provides a separate intervention on post-gating MLP activations. Each condition overwrites one selected neuron at every valid prefix position before

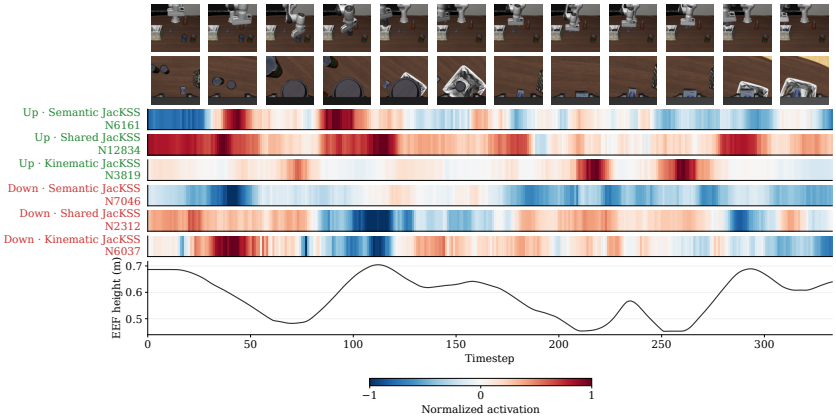


Figure 5: **Motor features track the movement.** Signed activations of 6 layer-6 neurons (semantic, shared, and physical for up and down) along an unedited held-out LIBERO-10 demonstration with two pick-and-place cycles, with camera frames on top and end-effector height at the bottom. Kinematic and shared neurons are phase-locked to lifts and descents; semantic neurons are broad. Rows are scaled independently, so colors compare along a row but not across rows. Neuron indices are in Appendix E.

the down projection. A separate unedited run provides the paired baseline. Clamp values are $\{0, 0.5, 1, 2, 4, 8, 16, 32\}$, so the zero-clamp condition suppresses the neuron and must not be used as the unedited control. The original figure caption specifies the 75-demonstration evaluation, paired noise, command conversion, and selected indices. We preserve that protocol rather than treating it as the 200-frame residual-swap experiment.

For a neuron activation a_i and write vector w_i, clamp, multiplication and addition would induce the following direct changes before subsequent blocks:

$$\Delta h_{\text{clamp}} = (\kappa - a_i)w_i, \qquad \Delta h_{\text{multiply}} = (\beta - 1)a_i w_i, \qquad \Delta h_{\text{add}} = \gamma w_i. \tag{8}$$

Their no-op settings are different: multiplication uses $\beta = 1$ and addition uses $\gamma = 0$, whereas a fixed clamp is not generally a no-op.

E.2 Qualitative evaluation for activations per timestep

Figure 5 follows selected layer-6 neurons through an unedited demonstration with two pick-and-place cycles. The kinematic up neuron is positive during both lifts, the kinematic down neuron is positive on the first descent and negative on the first lift, and the shared down neuron is negative while the object is carried and positive on the second descent; the semantic traces are broad and weakly phase-locked. Kinematic features rise and fall with the movement, whereas semantic features are present throughout. The example contains 334 timesteps of an unedited demonstration. Signed neuron activations are averaged over valid prefix positions and each row is scaled by its own 99th percentile of absolute activation. Colors are therefore comparable along a row but not in magnitude across neurons. The illustrated up kinematic neuron N3819 differs from the up clamp selection N5111; the illustration must not be treated as a trace of that clamp condition. Together these analyses complement the behavioral interventions without establishing a complete causal circuit from language to motion.

F Limitations and Future Work

JacKSS relies on local, first-order sensitivities, so adaptive or closed-loop steering that tunes strength and layer online is a natural extension. Scaling evaluation to more embodiments, larger real-world trial counts, and held-out tasks would further clarify how well the directions generalize. Speed and smoothness remain coupled, and object choice is steered only semantically; constrained objectives and differentiable surrogates, such as target end-effector positions, could enable finer and kinematically grounded control.